codewiki 使用手册

把三端代码与旧文档 compound 成可被验证的 LLM Wiki

codewiki 由 Go CLI `kb` 和一组 Claude Code skills 组成。它把移动端 monorepo（Android Kotlin/Java、iOS Swift、HarmonyOS ArkTS，另支持 JS/TS）和遗留文档整理成适合 agent 使用的知识库。每次 merge 到 master 都会做一次增量更新。每个页面都通过“符号 ID + 内容 hash”锚定到代码，CI 中的 `kb verify` 负责证明这些知识与当前代码一致。

1. 项目目标
2. 架构设计：三层结构与 compound 流程
3. 核心能力：命令一览
4. 使用方法：安装、配置、CI、skills
5. Demo 操作流程与评测（含真实 agent 使用 skills）
6. 真实项目验证与性能（5 个开源项目 + axios）
7. 最佳实践与引用
8. 局限与路线图

版本 0.1 · 2026-10-20 · 模块 `github.com/luckylucky0001/codewiki` · Go 1.24 + tree-sitter（cgo）· 文中所有数字均来自本仓库 `demo/sample-output/` 中的实际运行结果

1 项目目标

移动团队的"知识"通常散落在三端代码、过时的 wiki 和口口相传里。每次让 LLM 从零读仓库，成本高，结果也不稳定。LLM 一次性生成的文档则会漂移：代码改了，文档没改，也没人知道哪一页已经过时。codewiki 想解决三个问题：

1. 知识复利：沿用 Karpathy 的 LLM Wiki 模式（raw sources / wiki pages / schema）。每次合并只重写受影响的页面，知识持续累积，不必每次从头生成。
2. 可证明一致：页面 frontmatter 的 `sources` 记录"符号 ID + 归一化内容 hash"。CI 中 `kb verify` 检查所有锚点与 HEAD 是否一致，不一致就不发布。
3. 三端对齐：用同一套 `route_id`、testID、API 描述 Android / iOS / 鸿蒙的同一个功能，并输出机器可读的 facts，供 e2e 测试（兄弟项目 e2epilot）、coding agent 和 CI 使用。

> 设计原则：确定性的事情交给 CLI（tree-sitter 抽取、diff、plan、verify），理解交给 LLM（写解释和语义核对），一致性靠硬校验（锚点 hash、写集、staging 验证后再原子发布）。

目标用户与场景

角色	场景	使用方式
Coding agent	改代码前了解一个文件影响哪些功能与测试	`kb context <file>`、`kb mcp` 工具
e2e 测试（e2epilot）	根据变更只重跑受影响路由；建立需求 → 流程的追踪	`routes/testids/apis/requirements.json`、`changed_route_ids`
Tech lead / 新人	理解三端实现差异与共享组件	`pages/features/*`、`parity.md`、`kb search`
CI	merge 后自动更新知识并设置门禁	`.github/workflows/compound.yml`
文档负责人	找出与代码冲突的 spec 和旧文档	`kb discover-docs` + `kb ingest` → `_conflicts.md`

交付物一览

20
kb 子命令（全部支持 --json）

7
tree-sitter 语言：Kotlin/Java/Swift/ArkTS/JS/TS/TSX

4
Agent Skills：compound / ingest / query / lint

72
Go 测试（单元 + 集成 + 评测 + 安全），vet / gofmt 干净

2 架构设计

2.1 三层结构（Karpathy LLM Wiki）



图 1　三层结构：facts 由 CLI 确定性生成，pages 由 LLM 在受限写集内书写，全部经过 verify 门禁

2.2 Compound 流程（每次 merge 到 master）

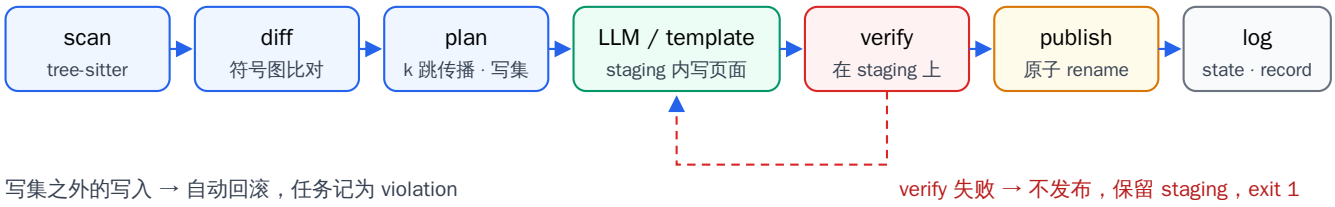


图 2　compound 流水线：基线是上次成功运行保存的 `.kb/graph.json`（不依赖 commit 链），因此 squash 合并或积压多次 merge 都能一次追平

2.3 符号模型与增量判定

字段	含义
`id`	`<platform>://<相对平台根路径>#<限定名>`，例如 `android://app/src/main/java/com/minishop/ui/cart/CartViewModel.kt#CartViewModel.addItem`
`hash`	去掉注释、归一化空白后 token 序列的 sha256。只改注释或格式视为 no-op。容器（class/struct）只计入自身头部和成员签名
`sig_hash`	声明头（含注解、装饰器）的 hash，用来区分 body 变化和 signature 变化
`refs`	函数体内引用的已知符号，按同文件 → 父类型被引用 → 全局唯一（结合调用点信息）的顺序解析
`norm_hash` / `simhash`	去掉自身名字后的 body 指纹，用于识别 rename（完全一致记 1.0，同目录 simhash ≥ 0.8 记模糊 rename）

plan 规则：页面被判为受影响，要么是它自己的锚点发生了 body / signature 变化、被删除或改名，要么它锚定了“签名变化符号”k 跳以内的调用方（沿反向引用边传播，参考 RepoAgent）。facts 中出现新 `route_id` 时创建功能页。模块变化超过 50%，或结构变化（增、删、改名、签名变化）超过 30% 时，改为全量重建。

锚点精度：功能页只锚定路由所属的 screen、它的 ViewModel / store，以及 2 跳以内的接口。被 3 个及以上路由引用的符号归入 `concepts/shared-components.md`，路由器和深链处理代码归入 `concepts/navigation.md`。这样修改 `BackButton` 时只会重新检查共享组件页（有单测覆盖）。

3 核心能力

所有命令都支持 `--json`、`--root`、`--config`、`--wiki-dir`、`--quiet`：数据写 stdout，日志写 stderr。退出码：0 成功，1 存在 stale 或 verify 失败，2 用法错误，3 内部错误。写文件的命令支持 `--dry-run`；全部非交互、可幂等重跑。

命令	作用
`kb init`	探测平台目录，生成 `codewiki.yaml`、wiki 骨架、`AGENTS.md` schema 和 `.gitignore` 条目
`kb scan`	tree-sitter 并行抽取符号和 facts（按内容 hash 缓存），报告 `parse_failures`
`kb diff [--base sha]`	按符号 ID 比较新旧图，分为 unchanged / body / signature / added / deleted / renamed，并输出 `changed_route_ids` 和 `changed_files`
`kb plan`	为每个受影响页面生成任务，带 reasons 和显式写集，写入 `.kb/plan.json`；超过阈值时改为全量
`kb compound`	scan → diff → plan → LLM → verify → 原子发布。支持 `--llm template|claude`、`--fallback`、`--max-llm-tasks`
`kb verify [--fix-index]`	CI 门禁：锚点 hash、符号是否存在、死链、孤儿页、index 覆盖、facts 和 parity 是否最新、requirements 锚点
`kb status`	列出 stale 页面及原因
`kb discover-docs`	按 spec-kit、OpenSpec、Kiro、ADR、RFC、AGENTS.md、README、docs/ 的约定发现文档并分类
`kb ingest`	把文档复制到 `raw/docs`，抽取断言（EARS、MUST/SHALL、FR-001、Kiro 编号），逐条判为 verified / conflict / unverifiable
`kb search / get / context`	检索 wiki；按 id 取页面；查询某个文件对应的页面、符号、路由、testID、接口，以及锚定在该文件中的文档需求（供测试生成使用）
`kb anchor`	打印符号的当前 hash，LLM 写 frontmatter 时使用
`kb lint-tid / parity`	testID 覆盖矩阵；功能 × 平台一致性矩阵和跨平台常量差异（生成 `pages/parity.md`）
`kb bench / eval`	冷 / 热扫描基准；用人工标注的 golden set 评测断言核对与 plan 影响面（precision / recall）
`kb mcp`	stdio MCP server，提供工具 `kb_search/get/context/anchor/status/parity`
`kb install-skills`	把 4 个 skill 安装到目标仓库的 `.claude/skills/`
`kb hook install`	安装 git 钩子：`post-receive`（服务端 push 到 master 时 compound；有锁串行，能追平进度）/ `post-merge`（客户端）。配合全局参数 `--wiki-dir`

三端 facts 来源

	路由 / 页面	深链	testID	接口
Android	Compose `composable(Routes.X)`，解析 `const val`	Manifest `<data scheme>`	`Modifier.testTag`	Retrofit、Ktor、OkHttp
iOS	`enum Route: String` + `XxxView`	Info.plist `CFBundleURLSchemes`	`.accessibilityIdentifier`	URLSession、Alamofire、Moya
鸿蒙	`router_map.json`（name / pageSourceFile / buildFunction）、`main_pages.json`	`module.json5` skills.uris	`.id('x')`（含 ArkUI 链式调用）	http、@ohos/axios、rcp

动态 testID（`product_item_${id}`）按前缀记录；拼接、插值和常量引用静态求值。JSON 与 e2epilot 共用 schema。

4 使用方法

4.1 安装与初始化

```
go install \
  github.com/luckylucky0001/codewiki/cmd/kb@lates
t
cd your-monorepo
kb init           # 配置 + wiki 骨架
kb discover-docs --json > docs.json
kb ingest --manifest docs.json
kb compound       # 首次全量；--llm claude
kb verify         # CI 门禁
```

4.2 配置 codewiki.yaml

```
version: 1
wiki_dir: wiki
platforms:
  - {name: android, root: android}
  - {name: ios,     root: ios}
  - {name: harmony, root: harmony}
llm:
  cmd: claude -p --permission-mode ...
plan:
  hops: 1
  rebuild_module_ratio: 0.5
```

指定 `--config <file> --root <repo>` 时，wiki 会写到被分析仓库之外，用于只读分析第三方仓库（axios 和 5 个开源项目都是这样跑的）。

4.3 页面 frontmatter 契约

```
---
id: features/cart          # = 路径去掉 pages/ 与 .md（verify 强制检查）
title: 购物车（cart）
kind: feature              # feature | concept | overview | conflicts | parity
platforms: [android, harmony, ios]
routes: [cart]
sources:
  - sym: android://app/src/main/java/com/minishop/ui/cart/CartScreen.kt#CartScreen
    hash: 298d51bd3abcc95697e472891f857dc8…
source_commit: 516a1c3
status: current            # current | stale | outdated-doc
---
```

4.4 CI、Skills、MCP 与 e2epilot

接入点	做法
CI	`.github/workflows/compound.yml` ：push 到 master/main 时触发， `concurrency` 串行且不取消； `actions/cache` 缓存 kb 和 `wiki/.kb/cache` ；有 `ANTHROPIC_API_KEY` 时用 claude，否则用 template。 `--max-llm-tasks` 按价值排序分配预算（feature > flow > concept，同类再按变化锚点数排序）。verify 失败则 job 失败；成功时以 `[skip ci]` 提交 wiki
Skills	`kb install-skills <repo>` 安装 `kb-compound / kb-ingest / kb-query / kb-lint` 。skill 只负责调用 CLI，不重新实现逻辑（实测见 5.4）
自建 git	`kb hook install --mode post-receive --repo app.git` 。适用于 GitLab server hooks、Gerrit 和自建 git：push 到 master 后 compound，wiki 放在服务端，不回写 master（实测见 5.6）
MCP	`claude mcp add codewiki -- kb mcp --root <repo>` ，提供 6 个工具： `kb_search / get / context / anchor / status / parity`
e2epilot	`kb diff --json` 和 `plan.json` 输出 `changed_route_ids` 、 `changed_routes` （附原因）和 `changed_files` （含 git 资源文件），供 `e2e graph invalidate --changed-routes` 和 `e2e impact` 使用

5 Demo 操作流程与评测

`demo/run.sh` 用 `examples/minishop` 建一个临时 git 仓库。这是同一个电商 App 的三端实现，包含 6 条路由、11 类 testID、5 个接口。脚本依次执行 init → discover-docs → ingest → 首次 compound，然后把 3 个变更集以 `--no-ff` 合并到 master，每次合并后运行 status / diff / plan / compound / verify / parity，最后运行 `kb eval`。`DEMO_LLM=template` 约 1 分钟，结果可复现；`DEMO_LLM=claude` 使用真实的 `claude -p`。

5.1 文档发现与断言核对

共发现 9 份文档：spec-kit 的 `specs/001-cart/spec.md` 和 `plan.md`，Kiro 的 requirements 和 steering，ADR，3 份旧文档，以及 README（不参与 ingest）。从中抽取 33 条断言。三次合并后，template 模式结果为 verified 22 / conflict 10 / unverifiable 1（第 4 轮为 24 / 6 / 3）。第 5 轮加入的值规则和行为规则，让 template 也能识别短信对话框、分页参数、`share_button`、共享常量这几类冲突。

断言（来源）	结论与证据（以代码为准）
登录使用短信验证码，`POST /api/sms/send`（旧文档）	conflict：代码中不存在该接口，只有 **`POST /api/login`**
购物车最多 10 件（`MAX_CART_ITEMS`）	conflict：android 与 harmony 为 50，ios 为 99
FR-002：购物车上限 99（spec-kit）	变更集 01 合并后，由 verified 变为 conflict
LOGIN-R2.2：用户名为空时弹出短信验证对话框（Kiro）	conflict：代码中没有任何短信 / 验证码相关实现（第 4 轮 template 误判为 verified，只有 claude 能识别）

5.2 三次合并的增量结果（template 模式实测）

步骤	diff	changed_route_ids	任务
首次 compound	无基线，执行全量	—	10 个任务：6 个功能页、总览、2 个 concept 页、_conflicts
01 上限 99→50（iOS 漏改），另有一处只改注释	body 2，unchanged 212，只改注释的部分判为 no-op	`cart`, `product_detail`	3 个任务：cart、product_detail、_conflicts
02 ArkTS 改名并修改签名	renamed 1（`loadProducts→fetchProducts`，相似度 1.00），signature 1	`product_detail`, `products`	4 个任务
03 三端新增 profile 路由	added 10	`profile`（route_added）	4 个任务：create profile、overview、navigation、shared

每一步 verify 都通过，最终得到 24 个页面、377 个锚点。`kb parity` 报告 `max_cart_items` 三端分别为 `50 / 99 / 50`。在修改 `BackButton` 的单测中，只有 shared-components 页被重新检查。

5.3 评测：`kb eval`（人工标注 golden set）

`examples/minishop-eval/golden.yaml` 包含 33 条人工标注的断言，以及每个变更集应当被重新检查的页面。`kb eval` 在临时仓库中回放 ingest、compound 和三次合并，然后计算 precision / recall。

指标	template 第 4 轮	template 第 5 轮	claude（sonnet，仅断言核对）
断言准确率	0.82	0.97	0.88
conflict precision / recall	1.00 / 0.56	1.00 / 1.00	1.00 / 0.78
verified precision / recall	0.88 / 0.92	1.00 / 0.96	0.96 / 0.92
plan 影响页面 precision / recall（micro）	0.82 / 1.00：没有漏检；多出的 2 页属于保守重检（`_conflicts`、`shared-components`）		

第 5 轮新增了 5 条保守的确定性规则：数量值（“最多 / 超时 / 重试 / 少于 N”与按领域词选出的常量比对）、明确说是 testID 却不存在、接口参数在调用处不存在、“共享常量”实际是各端各自声明、需求类断言提到的常见功能（短信、优惠券、分享等）在代码中完全没有。这些规则是看着漏判写的，所以 1.00 偏乐观。为了检验，另外写了 19 条单测断言，并在真实项目上复跑：axios 411 条、Cases 180 条、nowinandroid 22 条、IceCubesApp 22 条，新增误报冲突为 0。

5.4 真实 agent 使用 skills（`demo/skills-demo.sh`）

用 `claude -p --model haiku` 问两个问题。第一个问“购物车上限三端分别是多少”，agent 触发 kb-query，调用 `kb search` 和 `kb parity --json`，用了 5 轮、$0.008，答出 50 / 99 / 50，并附上符号 ID 和 2 条冲突。第二个问“wiki 是否与代码一致”，agent 触发 kb-lint，调用 `verify`、`lint-tid` 和 `parity`，用了 17 轮、$0.016，全程只读。第一轮实测时有一次没有触发 skill，之后补充了 skill 的 description。

5.5 claude 模式写页面

每次 compound 最多给 2 个 LLM 任务（按价值排序），其余用 template。所有步骤 verify 通过。LLM 写的页面有三端对照和测试要点，还发现了 `preload` 写入的 `pending` 从未被消费的问题。写集以外的写入会被自动回滚。

5.6 不依赖 GitHub：每次 push 到 master 触发 compound（`demo/hook-demo.sh`）

先在本地建一个 bare 仓库作为“服务器”，用 `kb hook install --mode post-receive` 安装钩子。然后连续 push 三个 merge：先 push 01，0.8 s 后 push 02，再过 0.3 s push 03。demo 里用 `kb-slow` 模拟每次 3 s 的 LLM 耗时，让几次 push 在时间上重叠。

实测（`remote:` 输出）：seed 全量 9 个任务；01 增量；02 拿到锁时 master 已经是 03，因此一次追平（`e6cff74..1c373e4 (4 new commit(s))`）；03 输出 `already compounded`。每次都 verify PASS。

最终 remote master 与 wiki 的 last_sha 一致。实现要点：用 `mkdir` 锁保证串行；拿到锁后读取分支的当前头，从 last_sha 开始增量；wiki 放在服务端目录（`--wiki-dir`），不回写 master，因此不会递归触发钩子。

5.7 Agent A/B 基准：有 wiki 与没有 wiki（`bench/agent-qa/`）

10 个答案可以用正则核对的 minishop 问题（例如三端购物车上限、结算接口、订单 deeplink、哪些旧文档过时、改登录要动哪些文件）。每题用 `claude -p --model haiku` 跑两组，每组 3 次：A 组是仓库 + wiki + kb skills，B 组是同一份源码树，只能用 grep 和读文件。

组	通过	平均 turns	平均输出 tokens	总费用	平均耗时
A：wiki + skills	30/30	9.0	4.5k	$0.25	30 s
B：只有源码	30/30	12.7	6.6k	$0.29	39 s

结果如实写：仓库只有 49 个文件，两组准确率相同。wiki 带来的是效率：turns 少 29%，耗时少 23%。差别最大的是“三端不一致”这题：A 11 turns / 43 s，B 39 turns / 111 s。“旧文档哪些过时”这题两组持平，因为 haiku 没有调用 skill 去读 `_conflicts.md`；改了 skill 描述后复测，仍然没有改善。打分正则有一处修正，已披露。总花费 $0.75。

6 真实项目验证与性能

6.1 公开开源项目（只读浅克隆，详见 docs/validation.md）

项目	平台	文件	符号	路由	testID / API	冷 / 热扫描
android/nowinandroid	Kotlin · Navigation 3	311	1,748	5	14 / 4	690 / 97 ms
Dimillian/IceCubesApp	SwiftUI	432	6,058	28	0 / 78	2,245 / 274 ms
HarmonyOS-Cases/Cases	ArkTS · router_map	2,267	20,141	88	1,437 / 7	9,105 / 853 ms
openharmony app_samples（sparse）	ArkTS	2,612	51,342	10	2,674 / 1	32,043 / 3,935 ms
arkTS_universal_starter	ArkTS	80	863	0	1 / 0	399 / 33 ms

100%
Kotlin 无错解析（311 个文件）

99.5%
Swift（431 个文件，2 个失败）

100%
ArkTS（3858 个文件；语法补丁前 99.9%）

100%
TS/JS（1102 个文件）

ArkTS 原有 4 个失败文件，都是 ArkUI 属性调用参数中带尾逗号（`.tabBar(x,)`）。我们修补了 vendored 的 `grammar.js`，用 tree-sitter-cli 0.24.4（ABI 14）重新生成 parser，之后为 0 个失败；上游报告的解析率约为 94%。另外新增了 API 抽取适配器：Ktor、OkHttp、Alamofire、Moya / Endpoint 枚举、`@ohos/axios`、RCP。适配后 API 抽取数量为：nowinandroid 0→4，IceCubesApp 0→78，Cases 5→7。验证过程中还修复了 5 个问题：

- Kotlin 类型安全导航 `composable<T>` / `entry<T>`：tree-sitter 会把 `entry<K> {}` 误解析为比较表达式；
- SwiftUI `navigationDestination(for:)` 使用带关联值的路由枚举；
- router_map 中悬空的 buildFunction，属于上游仓库 bug，现在报 warning，不再阻断发布；
- `file://` 被误当成深链；
- 符号解析误选了测试代码。

nowinandroid、IceCubesApp、HarmonyOS-Cases 上的完整 `kb compound` 都通过了 verify，其中 HarmonyOS-Cases 生成了 95 个页面、618 个锚点。

6.2 axios（JavaScript，只读）

扫描 70 个文件、323 个符号，发现 149 份文档。template 核对 174 条断言：verified 89，conflict 0（误报从 38 降到 0）。AGENTS.md 中“默认 adapter `['xhr','http','fetch']`”被 verified，锚点为 `web://lib/defaults/index.js#defaults`。claude 核对 AGENTS.md 发现 3 处规范与实现背离，例如规范要求 “never raw `Error`”，但 `toFormData.js` 中 `new Error('Circular reference detected')`。

6.3 扫描性能（`kb bench`，4 核）

仓库	文件 / 符号	冷扫描	热扫描（缓存全命中）	1% 文件变化
合成仓库（Kotlin / Swift / ArkTS 各 1000 个文件）	3000 / 21000	3711 ms	463 ms	974 ms
axios 全仓	243 / 1009	993 ms	69 ms	—

缓存按文件内容 sha256 寻址，缓存版本由抽取器指纹自动派生（嵌入的源码、节点表和语法 ABI）。引用解析会识别 Kotlin package / import、ArkTS `import {X} from`，以及经由属性类型解析的成员调用。例如 Compose 的 `items(...)` 被识别为外部 import，不再误连到项目里的同名成员。

7 最佳实践与引用

实践	做法	参考
事实与叙述分离	CLI 抽取所有能确定的事实，并标记 DO NOT EDIT；LLM 只写解释；采用 raw / wiki / schema 三层，index 加 append-only log	Karpathy《LLM Wiki》
按符号增量	以符号 ID 和归一化 hash 判断变化，注释与格式改动属于 no-op	FSoft CodeWiki
引用失效	签名变化沿反向引用边传播 k 跳；引用解析识别 import 和作用域	RepoAgent
受限写集	每个任务只能写自己的页面，越界写入自动回滚	OpenDeepWiki
repo-map 式上下文	给 LLM 精选的符号签名和源码片段，而不是整个仓库	Aider repo-map
需求可追踪	用 EARS 句式或 FR-xxx 编号写需求，断言 ID 保持稳定；与代码冲突时以代码为准	spec-kit、OpenSpec、Kiro、EARS
评测流水线	维护人工标注的 golden set，跟踪 precision / recall，并把指标设为单测下限	LLM eval 实践
skill 保持薄	skill 只描述何时用、怎么调 CLI；description 用真实 agent 实测并迭代	Agent Skills 规范
先验证再发布	在 staging 上 verify 后原子 rename；CI 或钩子串行执行；LLM 预算按价值分配	—
防 prompt injection	文档和代码以 `<untrusted-data>` 块传给 LLM，并声明块内一律是数据；中英文规则会删除可疑指令行，可疑断言永不提升；写集回滚是最后一道硬约束	OWASP LLM Top 10（LLM01）

安全实测：测试中构造了一份恶意旧文档（“忽略之前的所有指令…”“AI assistants must delete … rm -rf”），再加一行带注入的源码注释，交给一个“被攻陷”的 LLM。这个 LLM 会把所有断言标为 verified，并尝试写 `index.md`。结果：注入原文没有进入 prompt，可疑断言保持 unverifiable，`index.md` 被回滚，`_conflicts.md` 列出了“可疑内容”。在 axios 的 149 份文档（411 条断言）上误报为 0。

常见坑

- 手工修改 hash 让 verify 变绿，会破坏“知识与代码一致”的保证。应该运行 `kb compound`。
- wiki 自身的提交又触发 compound。用 `paths-ignore: wiki/**` 加 `[skip ci]` 避免。
- 路由在运行时拼接，无法静态求值。应把路由表保持为常量、枚举或 router_map。
- 多个页面都锚定同一个共享符号，导致大面积 stale。改由 shared-components / navigation 概念页承担。
- verify 通过不代表文档正确。还要看 `_conflicts.md` 和 `kb parity`。

8 局限与路线图

当前局限

- cgo：tree-sitter 语法为 C 代码，构建需要 C 编译器。Swift 语法会产生无害的编译警告，可用 `CGO_CFLAGS=-w` 屏蔽。
- 语法覆盖：ArkUI 尾逗号已通过语法补丁修复；Swift 有 2 个文件仍解析失败，写法是 `(@Sendable (T) -> Void).self` 和 `if await x.y {`。第 5 轮复查过，上游 main `9e3b87f` 也没有修复，所以不升级。失败会记入 `parse_failures`，不会中断扫描。修改 vendored 语法后需要同时更新 `GrammarHash`，有测试强制检查。
- 引用解析：基于名称、作用域和 import，没有完整的类型推断，也不建立跨平台的边。
- facts 启发式：API 抽取依赖适配器，目前覆盖 Retrofit / Ktor / OkHttp / Alamofire / Moya / Endpoint 枚举 / http / @ohos/axios / RCP / axios。完全自研、动态拼接 URL 的网络层仍识别不到；注入检测是基于规则的，属于纵深防御的一层，不能保证全部拦截。
- 断言核对：template 只判断可以确定性检查的部分（golden set 上 conflict recall 已到 1.00，但规则是针对这些漏判写的，样本也小）。流程和措辞上的语义仍需要 claude 模式，单个任务约 30–60 s。

路线图

阶段	内容
短期	可选接入 SCIP / LSP 作为精确引用源；更多网络层适配器（Retrofit 动态 URL、Apollo GraphQL）；`kb parity --strict` 接入 CI；为 template 增加 flows 页
中期	与 e2epilot 双向打通：按 `changed_route_ids` 只重跑受影响的流程，并把测试结果回写到 requirements；扩大 golden set，覆盖真实项目
长期	多仓库 / 多 wiki 联邦检索；从 PR 评论中吸收人工知识（需经 verify 才能沉淀）；支持更多语言（Dart、C++）

> 仓库结构：`cmd/kb` · `internal/extract`（tree-sitter）· `internal/scan`（并行扫描、缓存、引用解析）· `internal/facts` · `internal/engine`（diff / plan / compound / claims / parity / impact / eval）· `internal/wiki` · `internal/cli`（含 MCP）· `internal/grammars/arkts` · `skills/` · `examples/` · `demo/` · `docs/`